Methodology scheme: objects, formats and the scripts that make them

Companion to the methods draft (v3) and the path reference · state of the pipeline on 9 November 2026, 04:50 UTC, after the first end-to-end run · prepared for Soma · draft v3 for review

> **What this is.** The methods draft says *what* we computed and why. This scheme says *what has to exist* at each step, in what shape, and which of our scripts makes it. Each stage is written twice. First comes the **general process**, the operation any study doing this has to perform, written as a map between typed objects. Then comes **in our case**: the files, formats and scripts we actually used, and the marker that says the step is done. If you swap one of our scripts for another tool, the general line is the contract the replacement has to meet.
>
> Provenance marks as before: `Soma` decided by Soma, `read` read from a script or output, `derived` follows from the definitions, `inferred` a judgement. Participants are M/P codes only; <T> in a path is a placeholder for a treatment participant's folder name, never a real ID.

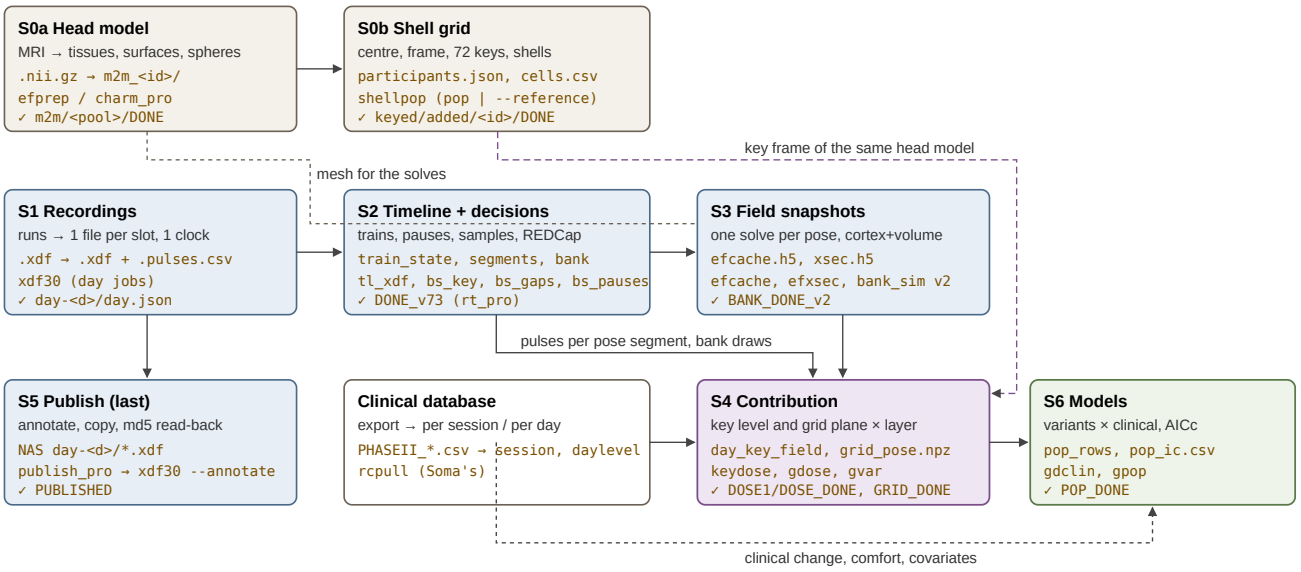
1. The scheme at a glance

The whole thing is one chain of maps. A stage can only run once its inputs exist, and it leaves a done marker when it finishes. That is the entire orchestration idea, and it's why the pipeline can be restarted at any point without redoing finished work.

$$\underbrace{\mathcal{I}_p \xrightarrow{\text{S0}} (\mathcal{H}_p, \mathcal{G}_p)}_{\text{anatomy}} \qquad \underbrace{\mathcal{R}_p \xrightarrow{\text{S1}} \mathcal{X}_p \xrightarrow{\text{S2}} (\mathcal{P}_p, \mathcal{Q}_p, \mathcal{C}_p) \xrightarrow{\text{S3}} \mathcal{E}_p}_{\text{treatment}} \qquad (\mathcal{G}_p, \mathcal{Q}_p, \mathcal{E}_p, \mathcal{C}_p) \xrightarrow{\text{S4}} X_p^m \xrightarrow{\text{S6}} (\text{tables, covariate maps, IC})$$

and S5 (publish the annotated recordings) runs last for each participant `Soma`. The chain has run end to end on the 12 phase-2 participants; the 20 phase-1 participants enter the same chain through a participant-set switch, with their own controller instances `read`. The objects:

Symbol	Object	Lives as
$\mathcal{I}_p$	structural images of participant p	NIfTI (`.nii.gz`)
$\mathcal{H}_p$	head model: labels, surfaces, registered spheres, cap	CHARM folder: NIfTI + GIFTI (`.gii`) + CSV
$\mathcal{G}_p$	key frame: centre, rotation, assignment, shell radii	JSON + CSV
$\mathcal{R}_p$	raw treatment and rest recordings	XDF (LSL), one per run
$\mathcal{X}_p$	one recording per slot, one clock, all pulses	XDF + per-file CSV/JSON
$\mathcal{P}_p$	pulse and train tables	CSV
$\mathcal{Q}_p$	coil pose per train (sampled / carried / stochastic draws)	CSV
$\mathcal{C}_p$	run-sheet and clinical values per session and day	CSV
$\mathcal{E}_p$	field snapshot per pose: cortex vectors, volume magnitudes	HDF5 (`.h5`) + NumPy
X_p^m	contribution grid of variant m on $(\varphi, \zeta, \text{layer})$, per day	NumPy archive (`.npz`) + CSV + GIFTI



Every box: one job per participant (S1: per participant-day; S6: one population job) under the batch controller. A job waits outside the CPU budget until its inputs' done markers exist, writes its outputs, then its own marker (✓). Rerunning a finished job is a no-op. Brown: anatomy · blue: treatment · purple: contribution · green: models. Monospace: file types, scripts, done markers.

Figure 1. Stages, artefacts and done markers. The arrows are data dependencies, which are also the order the batch controller enforces. S5 sits at the bottom on purpose: the annotated recordings go back to network storage only after the contribution is computed Soma.

2. Conventions every stage shares

Indices read. Participant p; treatment day $d \in \{1..5\}$, the rank of the local date (America/Chicago) among days with treatment runs; slot $j \in \{1..6\}$ from the run's file name; session $k = 6(d-1)+j$; train r within a session; pulse q within a train; file index $i = 13(d-1)+m$ with $m = 2j-1$ (rest before session j), $2j$ (session j), 13 (rest after session 6). Any table that has a session in it carries k, and any table that has a day carries d. That is what lets every later join be a plain key join.

Clocks read. Every time column says which clock it is on. *LSL* (the recorder's synchronised clock) is the authority for pulses. *Unix* is LSL plus the median file-name offset, so it is exact relative to other pulses and good to a few seconds in absolute terms. *Brainsight* local time is mapped to Unix by a whole-hour offset fitted from the data (UTC−5 here). A column without a clock is a bug.

Done markers. A stage's marker is an empty or small file written last, after every output is complete. A marker's presence is the only signal another stage may rely on; partial outputs without a marker are treated as absent read.

Read-only inputs. No stage writes into another stage's inputs. Raw recordings and the clinical export are never modified; re-running a stage rewrites only its own outputs read.

Identifiers. Working files keep the study's M/P codes. Archive brains get random pool numbers through a key file that never leaves the analysis machine. The treatment folder names are mapped to P codes through the study crosswalk at run time, and only P codes appear in figures and documents Soma.

3. S0: anatomy

3.1 Head model

General process. Each structural scan is turned into a tissue segmentation and cortical surfaces with a vertex-wise registration to a common sphere:

$$\mathcal{I}_p \;\mapsto\; \mathcal{H}_p = \big(L_p : \mathbb{Z}^3 \to \text{tissues},\;\; \{W, P, M\}_{p,h},\;\; \Sigma_{p,h},\;\; \{\vec{e}_{p,k}\}\big),$$

with L_p the label volume, W, P, M the white, pial and central meshes per hemisphere h, $\Sigma_{p,h}$ the registered sphere, and $\vec{e}_{p,k}$ cap electrode positions. Whatever tool is used, it must give meshes whose vertices correspond one-to-one across W, P, M and Σ, because area weights and registration are read per vertex derived.

In our case read. SimNIBS 4.6 CHARM, driven by Soma's preparation script for the population and by an archive driver for the 40 pool brains (42 jobs including a repeat scan). Input: T1 and T2 NIfTI from the non-defaced archive. Output: one `m2m_<id>/` folder holding the label volume (NIfTI), `surfaces/{lh,rh}.{white,pial,central,sphere.reg}.gii` (GIFTI), and `eeg_positions/*.csv` (template cap warped to the head). Marker: `m2m/<pool id>/DONE`. All 42 archive jobs finished with no failures read.

3.2 Key frame

General process. From $\mathcal{H}_p$ and a template $\{\bar{u}_k\}$:

$$\mathcal{G}_p = \Big(\vec{C}_p,\ M_p \in SO(3),\ \kappa_p : \mathbb{R}^3 \to \{1..72\},\ \rho_2, \rho_3 : \{1..72\} \to \mathbb{R}_+\Big),$$

with $\vec{C}_p$ the centroid of the pial-enclosed volume, M_p the anatomical frame (pole at the minimum of the smoothed outer radius, left–right from hemisphere centroids), $\kappa_p(\vec{x}) = \arg\max_g \pi(\vec{x}) \cdot M_p \bar{u}_g$ with the midline split, and the shell radii per key (methods v1, section 4). The template is computed once on a population and then frozen:

$$\bar{u}_k = \frac{\sum_{p \in \text{pop}} M_p^\top \hat{e}_{p,k}}{\|\cdot\|}, \qquad \text{new } p:\ \text{compute } (\vec{C}_p, M_p, \kappa_p, \rho) \text{ only.}$$

Any point of the head then has grid coordinates

$$\vec{u} = M_p^\top \frac{\vec{x} - \vec{C}_p}{\|\vec{x} - \vec{C}_p\|}, \quad \varphi = \operatorname{atan2}(u_y, u_x), \quad \zeta = u_z, \quad g = \kappa_p(\vec{x}), \quad t = \frac{\rho_2(g) - \|\vec{x} - \vec{C}_p\|}{\rho_2(g) - \rho_3(g)},$$

where t is the depth within the key's shell band: $t \le 0$ outside the cortex, $0 < t < 1$ inside the band, $t \ge 1$ below the deepest white surface read derived. **Every later stage maps data onto the grid through exactly this transform**, so it is the one thing that has to be identical everywhere.

In our case read. Population: `shellpop.py --grid apex` on the 29 head models → `pop_out6/` with `participants.json` (per participant: centre, rotation to the frame), `cells_all.csv` / `pop_cells.csv` (per participant × key: area, label fractions, folding index, ρ_2, ρ_3, crowns, fundi), `grid_topology.json` and `reference_cap.json` (the frozen template). New brains: `shellpop.py <m2m> --reference pop_out6 → keyed/added/<id>/{cells.csv, participant.json, check.json, DONE}`. All 42 archive jobs keyed; median per-brain label agreement with the population 0.81 read. Per-brain figures: `popvol.py → vol_<id>.png`, `plane_height_<id>.png`, staged by `keystage.py`.

4. S1: recordings to one file per slot

General process. Let $\mathcal{R}_{p,d,j} = \{f_1, \ldots, f_n\}$ be the runs of slot (d, j) ordered by first stamp. The stage produces one recording

$$\mathcal{X}_{p,i} = \text{concat}(f_1, \ldots, f_n) \ \oplus\ \text{annotation stream} \ \oplus\ \text{header record},$$

with three requirements Soma derived:

1. **Sample values unchanged.** Every original stream's samples are copied without resampling or filtering; only time stamps are put on one clock.
2. **Every join typed.** With $g = $ first sample of f_{l+1} minus last sample of f_l on the fastest stream, a join is *continuous* if $g < g_c$ and a *restart* otherwise, and both are marked in the annotation stream.
3. **Every pulse listed.** A pulse table with one row per TMS marker: $(\text{run}, r, q, t_{\text{LSL}}, t_{\text{unix}})$, numbered across continuous joins.

Clock check: $o_f = u_f - \tau_f$ (file-name time minus first synchronised stamp); a run with $|o_f - o_{f_1}| > \Delta$ is a clock reset and gets shifted and flagged; $t_{\text{unix}} = t_{\text{LSL}} + \text{median}_f\, o_f$.

In our case read. `xdf30.py`, one batch job per participant-day, $g_c = 1$ s, $\Delta = 60$ s, timezone America/Chicago. Raw runs are read in place from network storage; the work copy is staged on a separate local disk because the conversion is I/O-bound read Soma. Per slot, in `sub-pt-<T>/day-<d>/`:

File	Format	Contents
`<name>_s<j>.xdf`	XDF	all source streams on one clock; `PP-Annotations` (run start/end, MERGE_BREAK type + gap, CLOCK_SHIFT, first / 3000th / last pulse); `PP-Brainsight` added at S5; FileHeader holding the slot record (fixed 64 KB padding so it can be patched in place)
`.pulses.csv`	CSV	`run, train_k, pulse_in_train, pulse_n, t_lsl, t_unix, t_s` (+ `bs_pose, bs_state` at S5)
`.timeline.csv`	CSV	every marker sample + annotations on the merged clock
`.json`	JSON	sources, clock check, joins, per-stream counts and hashes, verification
`manifest.csv`, `day.json`	CSV, JSON	the day's slots with REDCap per session and clinical per day; md5 of every file; `day.json` written last = marker

Participant-level `pulses.csv, runs.csv, days.csv, manifest.csv` collect all days. Check a reader can do: each treatment slot has 3000 pulses = 75 × 40 unless aborted partial trains are listed read.

5. S2: timeline, clinical join and pose decisions

5.1 Trains, gaps, pauses

General process. From the pulse table, per session k and train r: $t_0(k,r)$, $t_1(k,r)$, $n_{k,r}$, and the gap $g_r = t_0(k,r) - t_1(k,r-1)$. A train starts a **pause** if $\mathrm{run}(r) \neq \mathrm{run}(r-1) \wedge g_r \geq g_c$, or $g_r > g_{\mathrm{pause}}$ Soma.

In our case read. `tl_xdf.py` builds a first timeline from the raw markers (`realtime_tl/<T>/{events,runs}.csv`) so the pose stage can start before conversion finishes; `bs_pauses.py` uses the converted LSL pulses when they exist and says which timing each row used (`timing = lsl | tl_xdf`). $g_{\mathrm{pause}} = 60$ s. `bs_gaps.py` keeps the exploratory gap table (`gaps.csv`). Population check, read from `train_state.csv` of all 12: 26 646 gaps, median 11.0 s, interquartile 8.3 to 14.7 s, 99th percentile 57.9 s, 257 pauses over 60 s; at least a tenth of gaps are 0.1 s (back-to-back or split trains, unchecked) read.

5.2 Neuronavigation samples

General process. Extract every recorded coil pose $(\mathbf{R}_i, \vec{x}_i)$ with its target $(\mathbf{R}_{\mathrm{ref},i}, \vec{x}_{\mathrm{ref},i})$ and time $t_{\mathrm{BS},i}$; fit the clock offset $h^* = \arg\max_h \#\{i : t_{\mathrm{BS},i} + 3600h \in W_{k(i)}\}$; express each sample as a deviation in the target frame,

$$\vec{d}_i = \left(\mathbf{R}_{\mathrm{ref},i}^{\top}(\vec{x}_i - \vec{x}_{\mathrm{ref},i}),\ \log(\mathbf{R}_{\mathrm{ref},i}^{\top}\mathbf{R}_i)^{\vee}\right) \in \mathbb{R}^6.$$

In our case read. Brainsight projects (`.bsproj`, a Core Data SQLite database; table of hand-taken samples, no per-pulse tracking) → Soma's `bspull` → `samples.csv` with `phase, sid, deid, name, local_time, day, slot, how, flag, m0..m15` (coil pose, row-major 4×4, mm) and `t0..t15` (target). The existing pull was used as is, never re-pulled Soma. `bs_key.py` → `samples_keyed.csv` (`idx, name, session_index, t_unix, position`), `bs_events.csv`, `summary.json`. The pose matrices used for simulation are the same samples in the head model's space, read from the field cache's `poses.csv` (`key, session, name, reference, m0..m15`).

5.3 Pose per train and the placement model

General process. Each train gets a pose in three layers: raw (latest sample by time), rule (sampled / sampled mid-run / carried / stochastic), and fill. The fill is a population model of deviations,

$$\vec{d}_{pi} = \vec{\mu} + \vec{b}_p + \vec{e}_{pi}, \quad \vec{b}_p \sim \mathcal{N}(0, \mathbf{T}), \quad \vec{e}_{pi} \sim t_\nu(0, \mathbf{S}),$$

with one bank $\{(\vec{z}_n, u_n)\}_{n=1}^{N}$ shared by all participants, so draw n is the same quantile for everyone Soma read. A stochastic segment s takes bank pose $\varpi_s(n)$ in draw n. The output of the stage, per participant, is a **weight tensor**

$$w_p[k, n, j] = \#\{\text{pulses of session } k \text{ delivered at pose } j \text{ in draw } n\},$$

which is everything the later stages need from the timeline. Any dose is then a contraction of w_p with field maps derived.

In our case read. `bs_pauses.py` v7.2, $N = 50$, fixed seed, ν by maximum likelihood over $\{3, 4, 6, 10, 30, \infty\}$. Outputs in `realtime_bs/<T>/`:

File	One row per	Key columns
`train_state.csv`	train	`session_k, train_k, timing, t_unix_first_pulse, t_unix_last_pulse, n_pulses, raw_*, state, bs_idx, pose_sample, segment_id, pause_id`
`segments.csv`	constant-pose stretch	`segment_id, session_k, t_start, state, pause_id, bs_idx, n_trains, n_pulses`
`pauses.csv`	pause	`session_k, pause_id, after_train, t_end, t_resume, gap_s, kind, rule, sample, rot_deg, shift_mm`
`dose/train_dose.csv`	train	`session_k, day, train_k, n_pulses, n_assumed, state, segment_id, pose_key, has_sim`
`dose/segment_draws.csv`	segment × draw	`segment_id, draw, bank_key`
`dose/poses_needed.csv`	bank pose to simulate	`bank_key, pid, reference, draw, m0..m15`
`popmodel/`	—	`placement_samples.csv`, `pop_params.json` (μ, $\mathbf{T}$, $\mathbf{S}$, ν), `zbank.csv`, `bank_all.csv`, `fill_check.csv` (held-out PIT)

`rt_pro.py` runs S2 per participant: `tl_xdf → bs_key → bs_gaps → bs_pauses`. Marker: `DONE_v73`. Trains still on file-name timing have no exact pulse count and are taken as 40 (`n_assumed`) until the LSL rerun read. The figure per participant is `tlplot.py` → `pose_timeline_<T>.png`.

5.4 Clinical join

General process. One table per session, $(p, k, d, O_k, f_k, \epsilon_k, \text{comfort items})$, and one per day, $(p, d, Y_{d,o}, Y_{d+1,o}, \Delta Y_{d,o})$, joined to everything else on (p, k) and (p, d) only, with date mismatches reported, never shifted.

In our case read. REDCap export `PHASEII_<date>.csv` → Soma's `rcpull.py` → `out/rcpull/phase2_1107/{session, daylevel, preday_self, preday_safety, endday, baseline, long}.csv`. Run-sheet values (output %, IAF, phase RMSE) also come from the IAF run-sheet export `CombinedPhases_REDCAP_IAF_<date>.csv`, which has the full session list. Pre-day event suffixes map to days 1–5 read.

6. S3: field snapshots

General process. For every pose j that carries pulses (recorded or bank), solve the quasi-static field once on the participant's head model with a fixed coil model and current slope $\dot{I}_0$:

$$\mathcal{E}_{p,j} = \Big(\vec{E}_j(v) \in \mathbb{R}^3,\ v \in \mathcal{M}_{\text{ROI}}\ ;\ \ \|\vec{E}_j(\vec{y})\|,\ \vec{y} \in \mathcal{B}_{\text{ref}(j)} \cap (\text{GM} \cup \text{WM}) \Big),$$

the cortical vectors on central-surface vertices of a region of interest, and the volume magnitudes on a grid box fixed to the pose's target. Since $\vec{E}$ is linear in $\dot{I}$, one factorisation of the system matrix per head serves every pose by back-substitution derived. A stage that adds poses must reproduce existing poses to numerical precision before its new fields are trusted.

In our case read. Soma's tools already held the recorded poses:

Store	Format	Layout
field cache (`efcache.py`)	HDF5	`poses/keys`, `poses/matsimnibs`, `E` (poses × N × 3, V/m at the cache's $\dot{I}_0$), `roi/{points, hemi, vertex_index, dist_to_ref}`; central-surface vertices within 70 mm of the targets
volume cross-sections (`efxsec.py`)	HDF5	per target `ref<k>/{frame, labels, t1, keep, poses, E}`: 1 mm box in the coil frame, $x, y \in [-40, 40]$, $z \in [-10, 60]$ mm, `E` as float16 magnitudes

Bank poses: `bank_sim.py` v2 with the same engine (one factorisation per head, back-substitution per pose), writing the same two layouts to `out/efcache_bank/<P>/{efcache.h5, xsec.h5, E/<bank_key>.npy, check.txt}`. The reference poses are re-solved as a check: on P116, 51 solves in 340 s, cortex relative RMS 9.9 × 10^{-8} against the cache and volume 2.1 × 10^{-4} (float16 storage) read. Driver `bank_pro.py`; marker `BANK_DONE_v2`. About 950 bank poses in total (50 per target) read.

7. S4: contribution

7.1 Pose maps on the grid

General process. Map each snapshot onto the grid of section 3.2 and average per bin:

$$\mathbf{m}_{p,j}(\ell, b) = \text{mean}\left\{ \|\vec{E}_j(\vec{y})\| : \vec{y} \in \text{layer } \ell,\ (\varphi, \zeta)(\vec{y}) \in \text{bin } b \right\},$$

with layer $\ell = 0$ the cortex (central-surface vertices, area-weighted) and $\ell = 1..4$ the volume by depth t: $[0, \frac{1}{3})$, $[\frac{1}{3}, \frac{2}{3})$, $[\frac{2}{3}, 1)$, $[1, \infty)$ read. The bins are rectangles in (φ, ζ), which are equal solid angles. Coverage $c(\ell, b)$ is the pulse fraction whose pose reaches the bin; bins below a coverage threshold are masked, not filled read.

7.2 Contribution variants

General process. The treatment is fixed: the same pulses, poses and fields. A **variant** m changes only how much each session counts, through a weight $\omega_m(k)$:

$$X^m_{p,d,n}(\ell, b) = \sum_{k \in d} \omega_m(k) \sum_j w_p[k, n, j]\ \mathbf{m}_{p,j}(\ell, b), \qquad X^m_{p,d} = \text{median}_n\, X^m_{p,d,n}.$$

This is the control design Soma asked for: the variants share everything except the bias in how regions receive their contribution, so any difference between them in explaining clinical change is attributable to that bias alone Soma derived. Because the fields and w_p are stored separately, a new variant is a new ω_m and costs no field computation derived.

Variants in our case read, with O_k the output (%), f_k the IAF, ϵ_k the RMSE read in Hz as in Soma's model $f \sim \mathcal{N}(f_k, \epsilon_k^2)$, $\bar{f}_p$ the participant's treatment-mean IAF, and one free tuning width σ:

Variant	$\omega_m(k)$	Meaning
m0 pulses	1	the pulses alone, as any protocol delivering them would
m1 output	$O_k/100$	field scales linearly with output
m2 sync	$\frac{O_k}{100}\,\frac{\sigma}{\sqrt{\sigma^2+\epsilon_k^2}}$	gain of a Gaussian tuning of width σ around the session IAF, averaged over $f \sim \mathcal{N}(f_k, \epsilon_k^2)$
m3 stable	$m2 \cdot \exp\Big(-\frac{(f_k-\bar{f}_p)^2}{2(\sigma^2+\epsilon_k^2)}\Big)$	the same tuning centred on the participant's mean IAF; sessions whose IAF drifted count less

The m2 gain is exact: $\mathbb{E}_{f\sim\mathcal{N}(f_k,\epsilon^2)}\left[e^{-(f-f_k)^2/2\sigma^2}\right] = \sigma/\sqrt{\sigma^2+\epsilon^2}$, a Gaussian convolution derived. Likewise centring the tuning on $\bar{f}_p$ multiplies it by $\exp(-(f_k-\bar{f}_p)^2/2(\sigma^2+\epsilon^2))$ derived. $\sigma \in \{0.5, 1, 2\}$ Hz, chosen per outcome and counted as a parameter read. Whether the run-sheet RMSE is in Hz or is a phase error in radians has not been settled. If it is a phase error, ϵ_k needs a conversion before m2 and m3 mean what they say inferred.

Second axis: IAF source. (f_k, ϵ_k) come either from the treatment run sheet or from an offline re-analysis of the same session's rest recording, occipitotemporal or frontal channels, with the spread over the recording as ϵ_k Soma read. Only m2 and m3 change. Trains, poses, draws and fields are shared, so all IAF sources are in the same sample space as the actual path Soma. Input: `~/Efields/out/iafcalc/iafw_sessions.csv` (`iafcalc.py`, `iafwhittle.py`), joined on participant, local date and slot, not on day rank read. Not run yet.

Further variants (specified, not built) Soma. Class 1, more variance without a protocol change: a wider pose spread, and the session weights permuted across days within a participant, $\omega^\pi(k) = \omega(\pi(k))$, to test temporal scrambling of the dose. Class 2, the contribution rotated to a nearby region on the sphere. And individual variants along the population modes of the shares. Permutations act on the weights only. The methods draft, section 12, gives the formal version.

7.3 Files

In our case read. Two levels exist side by side:

- **Key level** (`keydose.py`, drivers `dose1_pro.py`, `dose_pro.py` v3): `dose/*.csv` per pose, segment and day, markers `DOSE1_DONE`, `DOSE_DONE`. Its per-key trend readout is retired; the stage is kept because the grid stage reads `dose/train_dose.csv` from it read.
- **Grid level** (`gdose.py`, `gvar.py`; driver `grid_pro.py`), in `realtime_bs/<T>/grid/`:

File	Format	Contents
`grid_pose.npz`	NumPy archive	pose maps $\mathbf{m}_{p,j}$ (5 layers × bins, float16), `session_weight` $= w_p$, key plane map, key colours, key centres, ρ_2, ρ_3, cortical `convexity` (crown > 0, fundus < 0)
`grid_dose.npz`	NumPy archive	φ/ζ edges, layers, days, dose median / 5th / 95th percentile / mean over draws, coverage
`contrib.npz`	NumPy archive	variants, weights (variant × session), day and 5-day contribution per variant
`session_table.csv`, `session_weights.csv`	CSV	per session: day, pulses, output, IAF, RMSE, weight per variant, filled flags
`day_key_layer.csv`, `session_key_layer.csv`, `session_key_variant.csv`, `day_variant.csv`	CSV	the same quantities summed per key and layer, for tabular analysis and the EEG work
`vertex_bins.npz`	NumPy archive	every central-surface vertex → (bin, key, layer), so grid maps embed back onto the surface
`embed/<h>.contrib_<variant>_total.func.gii`	GIFTI	5-day cortex contribution on the surface, readable by any surface viewer
`embed/vol_*.nii.gz`	NIfTI	contribution on every grey- and white-matter voxel of the head model's tissue volume, via its key, bin and layer (`gembed.py`, marker `VOL_DONE`)
`griddose_*.png`, `griddose_vol*.png`	PNG	plane figures (`gdplot.py`, `gdclin.py`) and real-space slices (`gvol.py`, driver `gvol_pro.py`), flat folder, participant code in the name
`manifest.json`	JSON	inputs with md5, script version

Bins are 2° in φ by 0.02 in ζ, so 180 × 100 over the sphere read. Marker `GRID_DONE`. On P116 the covered bins were 19 950 (cortex), 11 855, 13 685, 15 110 and 12 490 (layers 1–4) read. Units are V/m × pulses at the cache's $\dot{I}_0$; only ratios are meaningful until the output scaling is confirmed inferred.

The `vertex_bins` map works in both directions. Any other spatial data that can be put on the head model, such as another imaging modality, can be binned onto the same plane with it and enter the models as a covariate or an outcome derived Soma.

8. S5: publication

General process. Write the decisions into the recordings, copy each day to network storage, read every copy back and compare checksums, write the day record last, and only then delete the local copy. This runs after S4 for each participant, so a slow network never holds up the analysis Soma.

In our case read. `publish_pro.py → xdf30.py --annotate --nas-out`, per participant, waiting on `DOSE_DONE` and the five local `day.json`. Each train's pose state and pose key go into a `PP-Brainsight` marker stream; stochastic trains are marked as such and no drawn pose is written as if recorded. Marker `PUBLISHED`, only when all five day records exist on network storage.

9. S6: models, export and covariate maps

9.1 Population model

General process. Rows are participant × day with a clinical change. Each variant's contribution is reduced to the **same low-dimensional basis**, fixed before any outcome is examined, and the variants are compared on **identical rows** by an information criterion:

$$\Delta Y_{p,d,o} = \alpha_p + \boldsymbol{\beta}^{\top}\mathbf{Z}\left(s^{m}_{p,d}\right) + c\,\log T^{m}_{p,d} + \gamma\,\mathrm{comfort}_{p,d} + \varepsilon, \qquad \mathrm{AICc} = -2\log\hat{L} + 2K + \frac{2K(K+1)}{n-K-1}.$$

In our case read. `gpop.py` (inside `grid_pro.py`, marker `POP_DONE`): $\mathbf{Z}$ = 3 principal components of the m0 shares over key × layer, the same loadings for every variant; 200 within-participant permutations. First run: 48 rows from 12 participants; m0 and m1 tie, m2 and m3 pay for σ, the spatial components do not earn their parameters (smallest permutation p 0.07).

9.2 Export

General process. For every participant with a finished grid, write one flat table per participant-day × variant × cell, with every covariate joined, and population reference tables, so that any later analysis reads tables, not arrays.

In our case read. `gexport.py`, run by `gvol_pro.py` after each new `GRID_DONE`: `data/participant/contrib_<T>.csv` (contribution, share, share against the population, key angles, pulses, output, IAF, RMSE, pose mix, comfort, clinical score / next / change) and `pose_<T>.csv` (each pose's coil orientation from Soma's Brainsight extraction, `samples_full.csv`, joined by session and sample rank, with its weight per day); `data/population/*` (share mean / SD per cell, model rows, IC table); `data/variants/<variant>/*` with a sample table per participant-day for the classifier. Marker `CONTRIB_EXPORT_DONE`.

9.3 Covariate maps

General process. For each covariate and each cell, a within-participant association between the cell's share and the covariate, corrected for the structure of the data: compositional shares, day trends shared by all cells, and many cells tested at once.

In our case read. `gexport.py` v4/v5 statistics: centred log-ratio shares; participant and day fixed effects removed from both sides; family-wise error by 1000 within-participant permutations of the covariate with the maximum $|r|$ over cells; ICC per cell; nested AICc baselines (participant effects, + total, + top key, + 3 components, + total and components). Per variant, $r_{\mathrm{variant}} - r_{\mathrm{m1}}$ per cell with a participant-bootstrap 95% interval and a Benjamini–Hochberg q. Maps drawn by `gvol.py --covmap` on the reference brain in AC-PC axes, into `~/spatial_sim/griddose_cov/`, marker `COV_DONE_<hash of the participant set>` so that a new participant reruns them. The first maps (participant-demeaned, uncorrected) are exploratory only; the v4 run had no output at writing.

9.4 Upload

Box `soma:Soma_DARPA/volumeGridRecon/OrgRealtime/contribution/{figures/{participant, population, covariates}, data/{participant, population, variants}}`; older content moved to `_superseded_20261109` read.

10. Orchestration

General process. Each stage is a driver that, asked for its plan, declares one job per participant (or participant-day) with its command, inputs, a memory estimate, a thread count and a done file. A controller admits a job when its inputs exist and the machine has room, keeps waiting jobs outside the CPU budget, and never reruns a job whose done file exists read.

In our case read. Soma's controller `bpro` on the analysis machine (23 cores, about 110 GB usable). A driver called with `--pro` prints a JSON spec, `{group, rows, participants: [{id, cmd, cwd, log, env, threads, rhat: {ram_gb}, inputs, markers, done_file}]}`. Several instances run side by side, each with its own run folder (`BPRO_BASE`) and memory cap (`BP_RAM_CAP_GB`), and every step is logged to `steps.csv`. Each stage runs as its own instance, separately for phase 1 and phase 2; the instance names are in the path reference. Every script also exits cleanly with a `waiting:` line when an input is missing, so a stage can be queued before its inputs exist.

Appendix A. Scripts, paths and markers

Script versions (md5, project copy, 04:50 UTC), every path each stage reads and writes, its done marker and its controller instance are in the companion **path reference** (`paths_reference.pdf`, and `paths.tsv` for scripts), which replaces the version table and folder tree of the earlier scheme. The methods

draft, Appendix A, lists the drivers and workers per stage.

References

- Burnham KP, Anderson DR (2002). *Model Selection and Multimodel Inference*, 2nd ed. Springer.
- Puonti O et al. (2020). Accurate and robust whole-head segmentation from magnetic resonance images for individualized head modeling. *NeuroImage* 219:117044.
- Thielscher A, Antunes A, Saturnino GB (2015). Field modeling for transcranial magnetic stimulation. *Conf Proc IEEE EMBC* 2015:222–225.